

UNIT – 3: SORTING ALGORITHMS

Course Details

Name of the School:

School of Engineering & Technology

Program (Class / Semester / Batch):

B.Tech (CSE / AIML)

Course Title:

Data Structures

Course Code:

ETCCDS202

Unit Number:

3

Unit Title:

Sorting Algorithms

No. of Hours:

10

Student Details

Student Name: Himalaya Singh

Roll No.: 2501730154

Section: A

Submission Date:

Submitted By

Himalaya Singh

Submitted To

Neetu Chauhan

Sorting Performance Analyzer (SPA) Report

1. Introduction

In this assignment, we implemented three sorting algorithms: **Insertion Sort**, **Merge Sort**, and **Quick Sort**. The objective was to analyze their performance on different types of datasets and compare practical results with theoretical time complexities.

2. Algorithms Implemented

- **Insertion Sort**
 - **Merge Sort**
 - **Quick Sort**
-

3. Dataset Description

The algorithms were tested on three types of datasets:

- Sorted Data
- Reverse Sorted Data
- Random Data

For each type, the sizes used were:

- 1000
 - 5000
 - 10000
-

4. Execution Time Results

Size	Type	Insertion (s)	Merge (s)	Quick (s)
1000	Sorted	0.000085	0.000971	0.000778
1000	Reverse	0.028958	0.001189	0.001248
1000	Random	0.013636	0.001113	0.000824
5000	Sorted	0.000331	0.004685	0.004560

Size	Type	Insertion (s)	Merge (s)	Quick (s)
5000	Reverse	0.595803	0.004385	0.004575
5000	Random	0.308367	0.006544	0.005248
10000	Sorted	0.000623	0.009091	0.009947
10000	Reverse	2.404939	0.008928	0.010222
10000	Random	1.230829	0.013675	0.010662

5. Analysis

a) Sorted Data

Insertion sort performs extremely fast on sorted data because it requires minimal comparisons and shifts. Merge sort and quick sort show consistent performance but are slightly slower due to recursive overhead.

b) Reverse Sorted Data

Insertion sort performs very poorly on reverse-sorted data, showing its worst-case behavior ($O(n^2)$). Merge sort and quick sort remain efficient and unaffected.

c) Random Data

Insertion sort again performs slower compared to the other algorithms. Merge sort and quick sort maintain good performance due to their $O(n \log n)$ complexity.

6. Comparison with Theory

Insertion sort shows quadratic growth in time as input size increases, confirming its $O(n^2)$ complexity. Merge sort and quick sort show significantly better scalability, matching their $O(n \log n)$ complexity.

7. Special Note on Quick Sort

Quick sort can degrade to $O(n^2)$ in worst-case scenarios, especially when a poor pivot (such as the last element in sorted data) is chosen. However, in this experiment, it performed efficiently for most cases.

8. Stability and Memory Usage

- **Insertion Sort:** Stable, in-place
 - **Merge Sort:** Stable, requires extra space
 - **Quick Sort:** Not stable, in-place
-

9. Conclusion

The experiment shows that insertion sort is suitable only for small or nearly sorted datasets. Merge sort and quick sort are more efficient and scalable for larger datasets. The practical results strongly align with theoretical time complexities.

10. References

- Course Notes
- Online resources (for understanding sorting algorithms)